FACULTY EMPHASIS · DEFINITIONS · COMPARISONS · DIAGRAMS · ALGORITHMS

> This file = Sections 4–8 of the final structure. Everything here is a **fast-access summary** of the detailed notes in files 01 and 02.

SECTION 4 — 🔥 FACULTY-EMPHASIZED / HIGH-PRIORITY TOPICS

4.1 Analysis of your BPIT assignments (facts read from the uploaded files)

Assignment-1 (Unit-I) — 7 sub-questions, 16 marks total, all CO1

Q	Type	Topic	Bloom	Marks
1a	Conceptual	Define DS + characteristics with examples	Understand	2
1b	Analytical	Client–Server, P2P, Grid, Cloud – architectures & applications compared	Analyze	2
1c	Problem solving	**Case study – Java RMI** (calculator/student info): architecture, remote interface, server, client, registry, steps, code, output	Analyze	3
2a	Conceptual	System models: need, Architectural & Fundamental with diagrams	Understand	2
2b	Analytical	Networking & internetworking; role of networks, internetworks, protocols	Analyze	2
3a	Conceptual	IPC: message passing vs shared memory (working, advantages, limitations, applications)	Understand	3
3b	Analytical	Distributed objects & RMI: remote objects, remote interfaces, stubs/proxies, invocation process	Analyze	2

Bloom split (Unit-I): Understand = 3 questions (7 marks); Analyze = 4 questions (9 marks). No "Remember" or "Apply/Create" level.

Assignment-2 (Unit-II) — 9 sub-questions, 32 marks total, CO2 and CO3

Q	Type	Topic	Bloom	Marks
1a	Conceptual	Synchronization; role of events & process states	Understand	2
1b	Conceptual	Happened-before (→) + Lamport clocks with example	Understand	3

Q	Type	Topic	Bloom	Marks
1c	Analytical	Need for physical clock sync; clock drift, skew, communication delay	Analyze	3
2a	Conceptual	Components of global state; importance of consistent global state	Understand	4
2b	Problem solving	**Case study – Distributed snapshot** with P1, P2, P3	Analyze	4
3a	Conceptual	Requirements of mutual exclusion + any two algorithms	Understand	4
3b	Analytical	**Election: Bully & Ring; compare messages & assumptions**	Analyze	4
4a	Conceptual	Consensus: agreement, validity, termination, example	Understand	4
4b	Problem solving	**Case study – Consensus with 5 processes, 1 may fail**; challenges	Analyze	4

Bloom split (Unit-II): Understand = 5 questions (17 marks); Analyze = 4 questions (15 marks). **Marks weight (Unit-II):** Coordination & agreement (3a, 3b, 4a, 4b) = **16 of 32**; Global state (2a, 2b) = **8**; Time/clocks (1a–1c) = **8**.

> Observation: the syllabus tags coordination/agreement as CO3 in your assignment, though the official CO3 text in the PDF is about Hadoop – this is a labelling quirk of the assignment; the *topics* are in the Unit-II syllabus, so study them.

4.2 Patterns noticed

1. **Every question is either "Explain/Describe…" (Understand) or "Explain + compare/case-study" (Analyze).** Expect *explain + diagram + example* type questions rather than pure one-liners.
2. **Case-study/problem-solving questions appear 3 times** (Java RMI, Distributed Snapshot, Consensus with 5 processes). Practise applying concepts to a scenario.
3. **Comparison is asked explicitly** in 1b (four architectures), 3a (message passing vs shared memory), 3b (Bully vs Ring).
4. **Diagrams/timelines/message-sequence diagrams are explicitly encouraged** in the Assignment-2 instructions.
5. **Higher marks (4 each)** are placed on global state, mutual exclusion, election, consensus – the "algorithm" part of Unit-II.
6. Nothing in the assignments on: *advantages of DS, RPC (explicitly), events & notifications, distributed debugging, multicast, vector clocks, logical-vs-physical table*. **They are in the official syllabus**, so they can still come (especially as short questions or in a compulsory objective question).

4.3 PRIORITY RANKING

(Priority is my inference from assignment weightage + official syllabus; it is not a prediction of the exact paper.)

🔥🔥🔥 VERY HIGH PRIORITY

Topic	Why
Java RMI case study (architecture, code, steps, registry)	Appears as a dedicated 3-mark case study (A1-1c) — highest single Unit-I mark — and RMI again in A1-3b; also in official syllabus "Case study – Java RMI"
RMI/Distributed objects, stub/proxy, remote interface (+ RMI vs RPC)	A1-3b + A1-1c; RPC is in the syllabus and is the natural comparison
Message passing vs shared memory	A1-3a explicitly asks comparison (3 marks)
Client-server vs P2P vs Grid vs Cloud	A1-1b explicitly asks comparison; syllabus lists all four
System models (Architectural + Fundamental, with diagrams)	A1-2a; syllabus
Lamport logical clocks + happened-before (numerical)	A2-1b; core numerical topic
Global state + Chandy-Lamport snapshot (P1,P2,P3)	A2-2a + A2-2b = 8 marks (25% of Unit-II assignment marks)
Distributed mutual exclusion (requirements + algorithms)	A2-3a (4 marks); syllabus
Election – Bully & Ring, comparison	A2-3b (4 marks)
Consensus (agreement/validity/termination, 5-process case study)	A2-4a + A2-4b = 8 marks

🔥🔥 HIGH PRIORITY

Topic	Why
Characteristics of DS + definition	A1-1a; base for all theory answers
Physical clock sync: skew, drift, delay, Cristian/Berkeley/NTP	A2-1c (3 marks); Cristian/Berkeley added as exam-oriented
Events & process states; synchronization intro	A2-1a
Networking & internetworking	A1-2b (2 marks)
IPC – sockets, request-reply, marshalling	supports IPC/RMI questions
Logical vs physical time	Natural comparison question; in syllabus ("logical time and logical clocks")
RPC (stub, steps, semantics)	In syllabus; needed for RMI vs RPC

🔥 MEDIUM PRIORITY

Topic	Why
Events and notifications (publish–subscribe)	Syllabus topic, no assignment question ⇒ likely short/2–3 mark

Topic	Why
Advantages of distributed systems	Syllabus topic; typically 1–2 mark or part of a definition question
Multicast communication (reliable, FIFO/causal/total ordering)	Syllabus topic, not in assignment ⇒ short/medium
Distributed debugging (possibly/definitely)	Syllabus topic, not in assignment ⇒ short
Vector clocks	Not named in syllabus (➕), but the natural improvement over Lamport; good for "compare" or tough-paper questions
Mutual exclusion: Lamport/Maekawa	Beyond "any two algorithms" — lab list includes Lamport ME

> ⚠️ These are indicators only. **Do not skip a medium-priority topic** — the term-end pattern in the handbook says Question 1 is compulsory, covers the *entire* syllabus and is made of short answers.

SECTION 5 — IMPORTANT DEFINITIONS (learn word-for-word)

Unit-I

Term	Definition
Distributed system	A system in which hardware/software components located at networked computers communicate and coordinate their actions only by passing messages.
Transparency	Concealing the separation/distribution of components from user and application programmer so the system is seen as a whole.
Concurrency	Multiple processes executing at the same time and sharing resources.
Scalability	Ability of a system to work effectively as the number of users/resources grows.
Client–server	Architecture where client processes request services from server processes that manage resources.
Peer-to-peer	Architecture in which all processes play similar roles, acting as both client and server, sharing resources without central control.
Grid computing	Coordinated sharing of geographically distributed, heterogeneous resources of multiple organisations to solve large problems as a virtual supercomputer.
Cloud computing	On-demand delivery of computing services over the Internet with pay-per-use and elastic resource pooling.
Middleware	Software layer between OS and applications that masks heterogeneity and provides a uniform programming model (RMI, RPC, CORBA).

Term	Definition
System model	An abstract, simplified description of relevant aspects of a distributed system.
Architectural model	Description of system structure in terms of components, placement and relationships.
Fundamental model	Abstract model describing common properties: interaction, failure and security.
Latency	Delay between sending of message by sender and its start of receipt at receiver.
Bandwidth	Total amount of information that can be transmitted in a unit of time.
Jitter	Variation in the time taken to deliver a series of messages.
Synchronous DS	System with known upper bounds on process execution time, message delay and clock drift.
Asynchronous DS	System with no bounds on execution speed, message delay or drift.
Omission failure	Process or channel fails to perform an action (crash, message loss).
Arbitrary (Byzantine) failure	Any type of error may occur including wrong values or malicious behaviour.
Internetwork	Network of networks connected by routers, functioning as one logical network.
IPC	Mechanisms allowing processes to exchange data and synchronise, by message passing or shared memory.
Marshalling	Converting data structures into a flat, transmittable byte format.
Distributed object	Object whose methods can be invoked remotely by processes on other computers.
Remote object reference	An identifier for a remote object valid throughout the DS.
Remote interface	Specification of the methods of an object that can be invoked remotely.
RMI	Invocation of a method of a remote object from another process, appearing like a local call.
RPC	Calling a procedure in another address space as if it were local.
Stub / Proxy	Client-side local representative that marshals the request and unmarshals the reply.
Skeleton / Dispatcher	Server-side component that receives request, unmarshals, calls servant.
Event / Notification	Occurrence of interest / message about that occurrence delivered to subscribers.
Publish–subscribe	Model where publishers emit events and an event service delivers them to interested subscribers.
Java RMI registry	Naming service (port 1099) binding names to remote object stubs.

Unit-II

Term	Definition
Synchronization	Coordinating actions of processes in time and order in the absence of a global clock/shared memory.
Event	An action (internal, send, receive) that changes a process's state.
Process state	Values of all variables of a process at an instant.
Happened-before (→)	Smallest relation with: same-process order; send → receive; transitivity.
Concurrent events	Events with no happened-before relation between them ($a \parallel b$).
Lamport clock	Software counter giving timestamps such that $a \to b \Rightarrow L(a) < L(b)$.
Vector clock	Array of counters (one per process) such that $a \to b \Leftrightarrow V(a) < V(b)$.
Clock skew	Difference between two clocks' readings at the same instant.
Clock drift	Rate at which a clock deviates from perfect reference time.
UTC	Coordinated Universal Time, standard derived from atomic clocks.
Cristian's algorithm	Client sets time = T_server + RTT/2.
Berkeley algorithm	Master averages the clocks of group and sends each an adjustment.
NTP	Hierarchical Internet time synchronisation protocol using strata.
Global state	Local states of all processes plus states of all channels (messages in transit).
Cut	A subset of system events; consistent cut has no orphan (receive-without-send) messages.
Consistent global state	One corresponding to a consistent cut — could have occurred in an execution.
Snapshot	A recorded consistent global state (Chandy–Lamport).
Marker	Special control message used to delimit a snapshot on channels.
Stable predicate	Property that once true stays true (deadlock, termination).
Possibly Φ / Definitely Φ	Φ true in some / every linearization's consistent global state.
Mutual exclusion	At most one process in critical section at a time.
Critical section	Code segment accessing shared resource.
Election	Choosing a unique coordinator among processes.
Multicast	Sending one message to all members of a group.
Reliable multicast	Multicast with integrity, validity, agreement.
Total order	All processes deliver messages in the same order.
Consensus	Processes with proposals agree on one value (termination, agreement, validity).
Byzantine generals problem	Agreement problem with possibly traitorous commander/lieutenants; needs $N \geq 3f+1$.
Interactive consistency	All processes agree on a vector of proposed values.

Term	Definition
FLP impossibility	No deterministic consensus in async system with even one crash failure.

SECTION 6 — IMPORTANT COMPARISONS (quick tables)

(Full versions of the 4 most important ones are in file 01/02: CS vs P2P vs Grid vs Cloud; Message passing vs Shared memory; RMI vs RPC; Bully vs Ring; Logical vs Physical time.)

6.1 Architectural vs Fundamental models

	Architectural	Fundamental
Focus	Components & placement	Abstract properties
Includes	client-server, P2P, proxy, mobile code	interaction, failure, security

6.2 Synchronous vs Asynchronous system

	Synchronous	Asynchronous
Time bounds	Known	None
Failure detection	Reliable via timeout	Impossible to be sure
Example	Real-time control	Internet

6.3 Message passing vs Shared memory (summary)

	Message passing	Shared memory
Works across machines	✔	✘ (DSM emulation)
Speed	slower	faster
Sync	implicit	explicit locks

6.4 TCP vs UDP

TCP	UDP
Reliable, ordered, connection	Unreliable, no order, connectionless

6.5 RMI vs RPC

RPC	RMI
Procedures	Objects/methods
No object passing	Passes objects, remote refs
Language-neutral (IDL)	Java-specific (Java RMI)

6.6 Local vs Remote invocation

Local	Remote
Same address space	Different processes/machines
Pointers work	No pointers; marshalling
No partial failure	Can fail (lost msg, crash)
Fast	Slow (network)
Exactly-once	maybe / at-least-once / at-most-once

6.7 Lamport vs Vector clock

Lamport	Vector
1 integer	N integers
$a \to b \Rightarrow L(a) < L(b)$	$a \to b \Leftrightarrow V(a) < V(b)$
No concurrency detection	Detects concurrency

6.8 Clock skew vs clock drift

Skew	Drift
Difference between two clocks at an instant	Rate of deviation of a clock from real time
Measured in seconds	Measured in s/s (ppm)
Result of drift	Cause of skew

6.9 Cristian vs Berkeley vs NTP

	Cristian	Berkeley	NTP
Sync type	External	Internal	External
Uses	time server	average by master	strata hierarchy
Scale	small	small	Internet

6.10 Logical vs Physical clocks

Logical	Physical
Order of events	Real time
No sync needed	Sync needed
Unaffected by drift	Affected

6.11 Consistent vs Inconsistent cut

Consistent	Inconsistent
For each receive in cut, send also in cut	Has receive whose send is outside (orphan)
Valid global state	Impossible state

6.12 Centralised vs Distributed vs Token-based ME

	Central	Ricart-Agrawala	Token ring
Messages	3	2(N−1)	1..N−1
SPOF	Yes	No coordinator but each process	Token loss

6.13 Bully vs Ring (summary)

	Bully	Ring
Messages best/worst	N−2 / O(N^2)	2N / 3N−1
Knowledge	all IDs	neighbour only
System	synchronous	asynchronous

6.14 Unicast vs Broadcast vs Multicast

Unicast	Broadcast	Multicast
1 → 1	1 → all	1 → group

6.15 FIFO vs Causal vs Total ordering

FIFO	Causal	Total
per-sender order	happened-before order	same order everywhere
sequence numbers	vector clocks	sequencer/ISIS

6.16 Crash vs Byzantine failure

Crash	Byzantine
Process stops	Behaves arbitrarily/maliciously
Need N ≥ 2f+1 (majority protocols)	Need N ≥ 3f+1
f+1 rounds (sync)	f+1 rounds (sync)

6.17 Stub vs Skeleton

Stub/Proxy	Skeleton
Client side	Server side
Marshals request	Unmarshals request, calls servant

SECTION 7 — IMPORTANT DIAGRAMS (draw-quickly versions)

D1. Distributed system

```
[Node A]──┐
[Node B]──┼── NETWORK (messages only) ──  looks like ONE system
[Node C]──┘
```

D2. Client–Server / P2P

```
C1─┐                    P1──P2
C2─┼─► SERVER           │ ╲╱ │
C3─┘                    P4──P3
```

D3. Software layers with middleware

```
Applications, services
Middleware (RMI, RPC, CORBA)
Operating system
Computer & network hardware      (Platform = OS + HW)
```

D4. Fundamental models

```
Interaction (latency, bandwidth, jitter, drift; sync/async)
Failure (omission, arbitrary, timing)
Security (threats: eavesdrop/tamper/replay/DoS; defence: crypto, auth, access control)
```

D5. RMI architecture

```
Client → Proxy → Comm.module ═net═ Comm.module → Dispatcher → Skeleton → Servant
         (marshal)   +Remote ref module                          (unmarshal, call)
```

D6. RPC

```
Client → Client stub ═ RPC runtime ═ Server stub → Server procedure
```

D7. Java RMI

```
Server: obj = new Impl(); registry.rebind("Name", obj)
Client: stub = registry.lookup("Name"); stub.method()
REGISTRY (1099) sits between both.
```

D8. Publish–subscribe

```
Publishers ─► EVENT SERVICE ─► Subscribers
```

D9. Happened-before timeline & Lamport timestamps (see 02 §5.5)

```
P1: a(1)──b(2)──────────────c(7)
P2: d(1)────e(3)──f(4)
P3: g(1)──────────h(5)──i(6)
```

D10. Consistent vs inconsistent cut

```
P1 ──●──●─┃─────
P2 ──●────┃─●──   send outside, receive inside = INCONSISTENT
```

D11. Chandy–Lamport snapshot (marker flow)

```
Initiator: record state → MARKER on all out-channels → record in-channels
Other:     1st MARKER → record state, chan=∅, send MARKERs, record other in-channels
           later MARKER → chan = messages recorded since
```

D12. Central-server ME

```
P ──request──► Coordinator ──grant──► P ─(CS)─ release ──►
```

D13. Token ring ME `P1→P2→P3→P4→P1` (token circulates)

D14. Ricart–Agrawala

```
Pi: WANTED → multicast REQUEST(T,i) → wait N−1 REPLIES → HELD → RELEASED → send deferred replies
```

D15. Bully message flow

```
P2 ELECTION→P3,P4,P5 ; OK from P3,P4 ; … ; P4 COORDINATOR→all
```

D16. Ring election

```
P1→P2→P3→P4→P1  ELECTION(id max so far) … own id returns → ELECTED circulates
```

D17. Reliable multicast/hold-back queue

```
Network → [hold-back queue] → delivery queue → application
```

D18. Consensus

```
proposals v1,v2,v3 → protocol → all decide same d
```

SECTION 8 — IMPORTANT ALGORITHMS (one-page pseudo-code)

A1. Lamport clock

```
Internal:  L = L + 1
Send:      L = L + 1; send(m, L)
Receive:   L = max(L, t) + 1
```

A2. Vector clock

```
Before event: V[i]++
Send: attach V
Receive (t): V[j] = max(V[j], t[j]) for all j; V[i]++
```

A3. Cristian

```
T0 = local time; ask server; get Ts; T1 = local time
new time = Ts + (T1 - T0)/2
```

A4. Berkeley

```
Master polls all → computes average offset (ignore outliers) → sends adjustment to each
```

A5. NTP offset

```
θ = ((t1 − t0) + (t2 − t3))/2 ;  δ = (t3 − t0) − (t2 − t1)
```

A6. Chandy–Lamport (see 02 §9.4)

A7. Central-server mutual exclusion

```
request → server: if free grant else queue;  release → server: grant to head of queue
```

A8. Token ring mutual exclusion

```
wait for token; enter CS; pass token to next (also if no need)
```

A9. Ricart–Agrawala

```
ENTER:  state=WANTED; multicast REQUEST(T,i); wait N−1 REPLY; state=HELD
ON REQUEST(Tj,j): if HELD or (WANTED and (Ti,i)<(Tj,j)) queue else REPLY
EXIT:   state=RELEASED; REPLY to all queued
```

A10. Lamport ME (REQUEST → REPLY → enter when head of queue & all replies → RELEASE) messages 3(N−1)

A11. Bully

```
detect failure: send ELECTION to higher ids
 if no OK within timeout → I am coordinator: send COORDINATOR to all
 else wait for COORDINATOR (higher process runs its own election)
```

A12. Ring election (Chang–Roberts)

```
start: participant=true; send ELECTION(myid)
recv ELECTION(id): id>myid → forward ; id<myid & !participant → send ELECTION(myid);
                   id<myid & participant → discard ; id==myid → I win, send ELECTED(myid)
```

A13. Reliable multicast (R-multicast)

```
B-multicast(g,m) ; on B-deliver(m) first time: if sender ≠ me B-multicast(g,m); R-deliver(m)
```

A14. Total ordering with sequencer

```
sender → sequencer: (m); sequencer assigns seq s, multicasts (m,s); receivers deliver in order of s
```

A15. Flooding consensus (crash, f failures)

```
for round 1..f+1: multicast newly learnt values; add received values
decide d = min(Values)
```

A16. Message complexity cheat-sheet

Algorithm	Messages
Central ME	3
Ring ME	1..N−1 (+ continuous)
Ricart–Agrawala	2(N−1)
Lamport ME	3(N−1)
Maekawa	≈3√N
Bully best / worst	N−2 / O(N²)
Ring election best / worst	2N / 3N−1
Chandy–Lamport	one marker per channel: up to N(N−1)
Flooding consensus	(f+1) rounds × O(N²)